

Course: Individual Software Development Process' 2026

*Iteration Report: **Iteration 1***

By **BigNiceMilk (BNM)**

Chosen Irl Challenge: Project C: Department Information & Communication Hub



Suwat	6810545956
Teiwasuculmetri	
Phunnipath	6810545816
Theankaew	
Apichai	6810545972
Pattanakamolkul	
Chayapol Kaewsakul	6810545557
Suppamok Tosranon	6810545921

Table of Contents

1. Project Snapshot.....	1
1.1 Project-Wide Gantt Chart.....	1
1.2 Current Iteration Gantt Chart.....	1
2. Sprint/Board Status Snapshot.....	1
3. Scope Addition/Change Log.....	2
4. Retrospective.....	2
5. Testing & Quality Notes.....	2
6. Project Risk Register.....	3
7. Individual Contribution Log.....	3

1. Project Snapshot

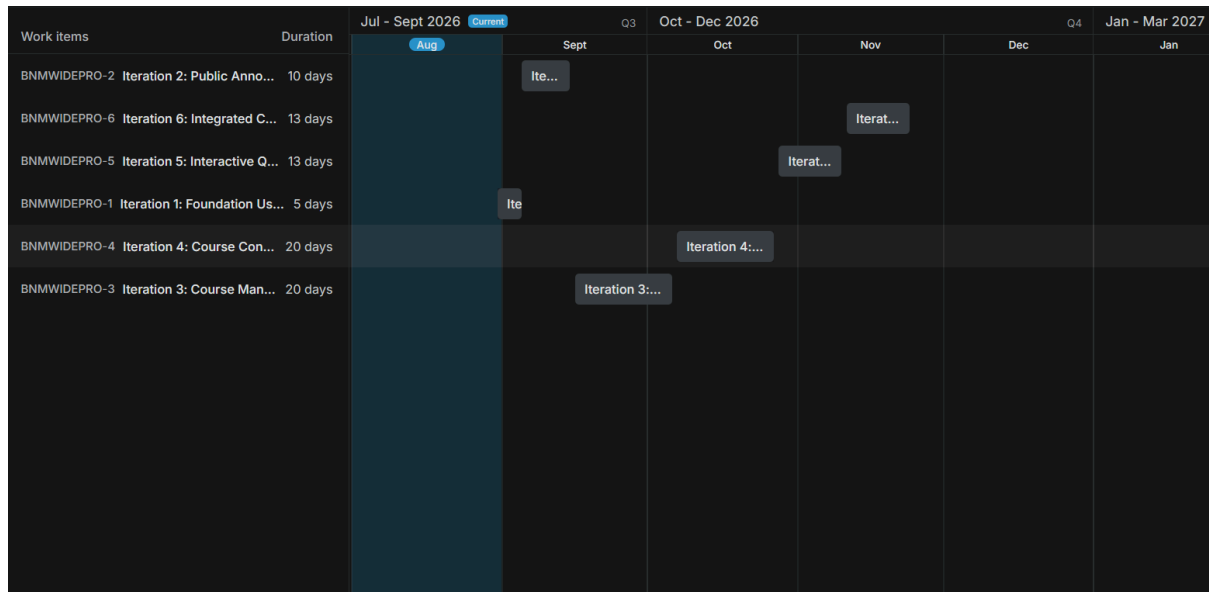
Note: This section must include both Gantt Chart for the whole project and the calculation of the critical path for the project, and each iteration. Created via Plane.so.

Note: Project Repository is mandatory for the report submission of Iteration 2. For Iteration 1, this can leave blank as “(WIP)”.

Project Repository:

<https://app.plane.so/isp1/projects/03771c9e-cf9a-4f4c-bccf-34ceedd4371b/issues>

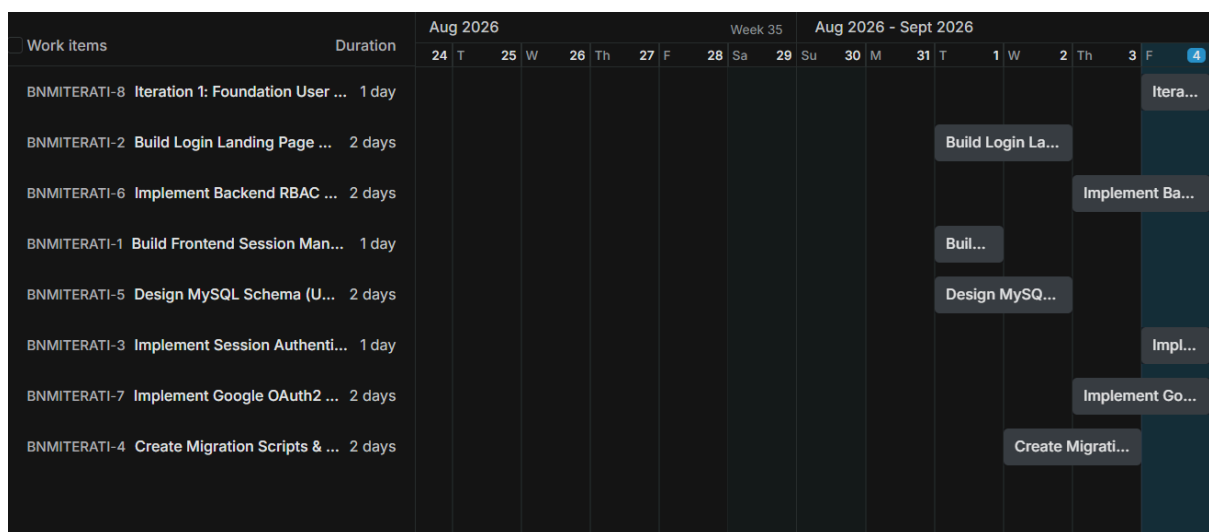
1.1 Project-Wide Gantt Chart



Critical Path: User Auth & RBAC Schema (IT-1) → Course & Class Code Engine (IT-3) → Quick Post & Ownership Guard (IT-4) → Real-Time Group & Q&A Engine (IT-5) → Calendar Feed & System Integration (IT-6)

Critical Path Duration: 98 calendar days

1.2 Current Iteration Gantt Chart



Critical Path: Finalize User & Role Database Schema → Implement Google OAuth2 Authentication with @ku.th Domain Guard (US-01) → Implement Session Authentication & Role-Based Access Control (US-05) → Deliver Login Page & Session Guard UI (US-01, US-05)
Critical Path Duration: 7 calendar days

2. Sprint/Board Status Snapshot

Note: User Stories Must Be Consistent User Stories in Software Proposal (Week 4). For Maximum Capacity , It Must Be:

- 2 User Stories for Iteration 1

Iteration Goal: Delivering MVP (Minimum Viable Product) and designs UI for login and registration page with Google OAuth2 with @ku.th and also implement role management for administrator users.

Capacity Declared: 2 user stories

User Story	Description	Owner	Status	Within Capacity (Y/N)	Notes
US-1	As a system user, I want to log in using my Google @ku.th account, so that I can securely access department and course resources.	Apichai, Phunnipath, Suwat, Suppamok, Chayapol	DONE	Y	Workflow focused first
US-5	As the Department Admin, I want to assign/edit user roles and manage all global announcements or lower-level lectures, so that overall system integrity is maintained.	Apichai, Chayapol, Phunnipath, Suwat, Suppamok	DONE	Y	RBAC will be prioritized first, superuser (admin dept) will be prioritized

3. Scope Addition/Change Log

Note: Log any item added, expanded, or swapped in after planning began. Empty is a good sign; frequent entries signal scope creep.

Item added/changed	Reason	What as dropped to compensate (one-in-one-out)
Migrated from MySQL to PostgreSQL	When using docker ,PostgreSQL is the default database model to use , for easy implementation and current routing , using default one is considered preferable.	MySQL Database (<i>dropped standard local/standalone MySQL instance</i>)
FAQ page and Forgot-password page added to the sign-in flow	The login card's "Having Problems?" sheet needed somewhere to lead. Both are UI surfaces for US-01, not new stories.	Password-reset <i>sending</i> deferred. The page validates the address and confirms the request but emails nothing, it deliberately does not claim a message was sent.
Sign-in ID field widened to accept non-numeric IDs	The seeded database uses A0001 and L0001 for the department admin and lecturer. The ten-digit rule locked both roles out of the product entirely.	The strict ten-digit rule now applies at <i>registration</i> only, where a student ID is actually being created.

4. Retrospective

What went well	What didn't go well	Action items for next sprint
Data model was implemented satisfied user requirement and ready for iteration 1 implementation	Some data models such as nisit number won't have constraint on 681,671 ligit nisit id base. It can input 123456781 and could go through the system	Allocate more time in later sprint for security.
Registration module has worked beautifully satisfied page redirect workflow (for new registration) and for logged in user	None , the feature currently satisfied the sprint	None , the feature currently satisfied the sprint
Sign-in, registration and dashboard UI delivered and merged inside the iteration (PRs #2, #5, #6, #7).	The UI was built against <i>assumed</i> data rules instead of the real schema. A ten-digit Nisit ID rule I added blocked the admin and lecturer accounts from signing in, and undoing it meant changes in three separate layers, form validator, HTML widget pattern, and the JavaScript that strips non-digits as you type.	Read the schema and seed data before writing any validation rule. Confirm which ID formats real accounts use.
Building on one base template and one stylesheet meant the FAQ and forgot-password pages cost very little to add once the system existed.	Multi-line {# ... #} comments are not comments in Django. Blocks of developer notes rendered onto every page above the sign-in card, and it went unnoticed until the app was actually run.	Run every page in Docker before calling it done. Static previews and template review did not catch either defect above.
The branch > pull request > review > merge workflow held. No commit went to main directly, and merge conflicts with back-end work stayed limited to a single line of requirements.txt.	The static previews drifted out of date on the very first stylesheet change after they were written.	Decide on the previews: drop them now that Docker runs the real pages, or accept the manual re-sync as a standing cost.
Backend authentication for login and registration was	Adding role-related changes to the existing user model	Coordinate database model changes before creating

successfully implemented using the custom user model. Role support was also added to distinguish users and provide a foundation for role-based authorization.	caused migration conflicts when changes from different branches were merged. Resolving the migrations and synchronizing the database took additional development time. Role restrictions also required further integration and testing.	migrations, synchronize feature branches with main more frequently, and add tests for each role to verify that users can only access authorized functions.
Deploy a docker for controlling environment and database for the project	None , the feature currently satisfied the sprint	None , the feature currently satisfied the sprint
Make Oauth2 working with the website and register in the database	Oauth2 needs a secret key and I do not know what is the best way to implement a secret key in the code.	None , the feature currently satisfied the sprint

5. Testing & Quality Notes

Manual testing was performed on the login, registration and dashboard pages across desktop and 375-pixel mobile width, in both signed-in and signed-out states, and five defects were found. Three concern input handling: the first name, last name and password fields all accept leading and trailing spaces without any feedback, and the server then silently trims them, so " Somchai " is stored as "Somchai", harmless on the two name fields, but on the password it means the credential saved is not the one that was typed, and a password containing deliberate spaces cannot be set; the fix is to trim and warn on the name fields and to reject surrounding whitespace on the password. The remaining two are unimplemented interface rather than faults in working code: the Forgot Password page validates the KU address and confirms the request but sends no email, because no mail backend is configured, so it is deferred to a future iteration and deliberately does not claim a message was sent; and the Help link in the top bar points at nothing, looks clickable, and will be removed. Defects found and fixed during development are recorded in the retrospective rather than repeated here.

6. Project Risk Register

Risk ID	Description	Likelihood (L/M/H)	Impact (L/M/H)	Mitigation/Contingency	Status
R-01	Nisit ID validation crack hole	M	L	Put on validation system via either on front end or back end	Monitoring
R-02	Hardcoded secrets in settings.py —	H	H	Move SECRET_KEY and DB	Open

	SECRET_KEY and DB password (postgres/postgres) are committed in plaintext, DEBUG=True, ALLOWED_HOSTS empty			credentials into .env before any non-local deployment; set DEBUG=False and populate ALLOWED_HOSTS	
R-03	Incomplete password-reset flow — the "forgot password" form confirms submission but no email is actually sent (no email backend configured)	H	M	Wire up a Django email backend before demo, or document self-service reset as out-of-scope with an admin-driven manual reset as fallback	Open
R-04	Unmanaged legacy schema — the Users model uses managed=False with a custom Postgres schema (search_path), so Django migrations don't track it; db_backup.sql and models.py can silently drift apart	M	H	Keep db_backup.sql and models.py manually in sync on every schema change; add a login smoke test after each deploy	Monitoring
R-05	Unmanaged legacy schema — the Users model uses managed=False with a custom Postgres schema (search_path), so Django	M	H	Keep db_backup.sql and models.py manually in sync on every schema change; add a login smoke test after each deploy	Monitoring

	migrations don't track it; <code>db_backup.sql</code> and <code>models.py</code> can silently drift apart				
R-06	Google OAuth login depends on an external service and correct env config — if <code>GOOGLE_OAUTH_CLIENT_ID/SECRET</code> are unset or Google is unreachable, that sign-in path fails silently	L	M	Keep the Nisit-ID/password login fully independent as the tested fallback; verify OAuth credentials before each deployment	Monitoring

7. Individual Contribution Log

Note: One row per member, filled in by that member. The submission of this file considered that the form below is consented and confirmed by the whole team. This section directly supports the Individual Project Contribution and Peer Evaluation grading components.

Note 2: For Iteration 1, evidence can be a reference to the video presentation/iteration report that is related to the responsible task(s) (e.g. Lalita's Evidence). However, for Iteration 2, the reference must be on **GitHub file and commit only, which is uploaded by that team member**.

Student Name	Tasks owned this sprint	Course concept(s) applied	Evidence	Self-rated effort (1-5)
Phunnipath Theankaew	Design MySQL Schema (Users Roles Permissions), Create Migration Scripts & Initial Seeds for Roles	Table mapping from the database with login/registration and admin dashboard; applied CRUD principles to the Create / update user roles	namelist/accounts/model.py	4
Suwat Teiwasuculmetri	Design MySQL Schema (Users Roles Permissions), Create Migration Scripts & Initial	Table mapping from the database with login/registration and admin dashboard; applied	namelist/accounts/model.py	4

	Seeds for Roles	CRUD principles to the Create / update user roles		
Apichai Pattanakamolkul	Deliver Login Page & Session Guard UI (US-01, US-05): design and build the sign-in, registration, dashboard, FAQ and forgot-password pages, the shared stylesheet, and the signed-in bar with role display.	Work as a team, and separating work. Managing a process for work to be completed in no time.	PR #2, #5, #6, #7 name_list/accounts/templates/accounts/name_list/accounts/static/accounts/css/style.css	4.67
Chayapol Kaewsakul	Implement backend authentication for Login and Registration; implement user role support and role-based access control for different user types.	Credential-Based Security, RBAC, MVC Architecture, and CRUD for user authentication and access control.	accounts/views.py accounts/forms.py accounts/models.py accounts/urls.py	4
Suppamok Tosranon	Design and build OAuth2 system to work with the website. Including registering new users when they login with OAuth2.	Splitting work with Chayapol, working mainly on the backend side(Making OAuth2).	namelist/views.py namelist/forms.py	4.20